

TiDB: 基于 Raft 的 HTAP 数据库

黄东旭、刘奇、崔秋、方珠河*、马晓宇、徐飞、沉力、刘唐、周宇兴、黄梦龙、万伟、刘聪、张建、李建军、吴雪莲、宋凌宇、孙若曦、于帅鹏、赵雷、尼古拉斯·卡梅伦、裴立泉、唐鑫 *PingCAP*

{黄、刘琪、崔秋、方竹和、马小宇、徐飞、沉力、tl、z、孟龙、伟万、刘聪、张建、杰伊、吴雪莲、松岭宇、孙若溪、yusp、zhaolei、nick、liquanpei、tangxin}@pingcap.com

抽象的

混合事务和分析处理 (HTAP) 数据库需要单独处理事务和分析查询, 以消除它们之间的干扰。为了实现这一点, 有必要维护为两种类型的查询指定的不同数据副本。然而, 为存储系统中的分布式副本提供一致的视图是一项挑战, 其中分析请求可以从大规模事务工作负载中高效地读取一致且新鲜的数据, 并且具有高可用性。

为了应对这一挑战, 我们建议扩展基于复制状态机的共识算法, 为 HTAP 工作负载提供一致的副本。基于这个新颖的想法, 我们提出了一个基于 Raft 的 HTAP 数据库: TiDB。在数据库方面, 我们设计了一个多Raft存储系统, 由行存储和列存储组成。行存储是基于Raft算法构建的。它可以扩展并实现事务请求的更新, 并具有高可用性。特别是, 它将 Raft 日志异步复制到学习器, 将元组的行格式转换为列格式, 形成实时可更新的列存储。该列存储允许分析查询有效地读取新鲜且一致的数据, 并与行存储上的事务强隔离。基于这个存储系统, 我们构建了一个SQL引擎来处理大规模分布式事务和昂贵的分析查询。SQL 引擎以最佳方式访问数据的行格式和列格式副本。我们还提供了强大的分析引擎 TiSpark, 以帮助 TiDB 连接到 Hadoop 生态系统。综合实验表明, TiDB 在针对 HTAP 工作负载的基准 CH-benCHmark 下实现了隔离高性能。

PVLDB参考格式:

黄东旭、刘奇、崔秋、方柱河、马晓宇、徐飞、沉力、刘唐、周宇兴、黄梦龙、万伟、刘聪、张健、李建军、吴雪莲、宋凌宇、孙若曦、于帅鹏、赵雷、尼古拉斯·卡梅伦、裴利泉、唐鑫。TiDB: 基于 Raft 的 HTAP 数据库。 *PVLDB*, 13(12): 3072-3084, 2020. DOI: <https://doi.org/10.14778/3415478.3415535>

1. 简介

关系数据库管理系统 (RDBMS) 因其关系模型、强大的事务保证和

*Zhuhe Fang is the corresponding author.

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 13, No. 12

ISSN 2150-8097.

DOI: <https://doi.org/10.14778/3415478.3415535>

SQL 接口。它们广泛应用于传统应用程序, 例如业务系统。然而, 旧的 RDBMS 不提供可扩展性和高可用性。因此, 在2000年代初[11], 互联网应用程序首选NoSQL系统, 例如Google Bigtable [12]和DynamoDB [36]。NoSQL 系统放宽了一致性要求, 并提供了高可扩展性和替代数据模型, 例如键值对、图形和文档。然而, 许多应用程序还需要强事务、数据一致性和SQL接口, 因此出现了NewSQL系统。像 CockroachDB [38] 和 Google Spanner [14] 这样的 NewSQL 系统为在线事务处理 (OLTP) 读/写工作负载提供了 NoSQL 的高可扩展性, 并且仍然保持事务的 ACID 保证 [32]。此外, 与许多 SQL-on-Hadoop 系统一样, 基于 SQL 的在线分析处理 (OLAP) 系统正在快速开发[16]。

这些系统遵循“一刀切”范式 [37], 针对 OLAP 和 OLTP 的不同目的使用不同的数据模型和技术。然而, 多个系统的开发、部署和维护成本非常昂贵。此外, 实时分析最新版本的数据也很引人注目。这在工业界和学术界催生了混合 OLTP 和 OLAP (HTAP) 系统 [30]。HTAP系统应该像NewSQL系统一样实现可扩展性、高可用性和跨国一致性。此外, HTAP系统需要高效地读取最新数据, 以保证OLTP和OLAP请求的吞吐量和延迟, 这两个附加要求是: *freshness*和*isolation*。

新鲜度意味着分析查询如何处理最近的数据[34]。实时分析最新数据具有巨大的商业价值。但在某些 HTAP 解决方案中, 例如基于提取-转换-加载 (ETL) 处理的解决方案, 并不能保证这一点。通过ETL过程, OLTP系统定期刷新一批最新数据到OLAP系统。ETL需要花费数小时或数天的时间, 因此无法提供实时分析。ETL 阶段可以通过将最新更新流式传输到 OLAP 系统来取代, 以减少同步时间。然而, 由于这两种方法缺乏全局数据治理模型, 因此考虑一致性语义更加复杂。与多个系统的接口会带来额外的开销。

隔离是指保证单独的 OLTP 和 OLAP 查询的隔离性能。一些内存数据库 (例如 HyPer [18]) 允许分析查询从同一服务器上的事务处理中读取最新版本的数据。尽管这种方法提供了新鲜数据, 但它无法同时实现 OLTP 和 OLAP 的高性能。这是由于数据同步损失和工作负载干扰造成的。[34] 通过运行 CH-benCHmark [13] (Hyper 和 SAP HANA 上的 HTAP 基准) 来研究这种影响。研究发现, 当系统共同运行分析查询时, 其可达到的最大 OLTP 吞吐量为

明显减少。SAP HANA [22] 吞吐量减少了至少三倍，HyPer 吞吐量减少了至少五倍。MemSQL [24] 也证实了类似的结果。此外，如果内存数据库仅部署在单台服务器上，则无法提供高可用性和可扩展性。

为了保证性能隔离，需要在不同的硬件资源上运行 OLTP 和 OLAP 请求。最根本的困难是在单个系统中维护来自 OLTP 工作负载的 OLAP 请求的最新副本。此外，系统需要保持更多复制之间的数据一致性。请注意，为了可用性还需要维护一致的副本[29]。高可用性可以使用众所周知的共识算法来实现，例如 Paxos [20] 和 Raft [29]。它们基于复制状态机来同步副本。可以扩展这些共识算法，为 HTAP 工作负载提供一致的副本。据我们所知，这个想法以前从未被研究过。

遵循这个想法，我们提出了一个基于 Raft 的 HTAP 数据库：TiDB。它为 Raft 共识算法引入了专用节点（称为 *learners*）。学习者从领导节点异步复制事务日志，为 OLAP 查询构建新的副本。特别是，学习者将日志中的行格式元组转换为列格式，以便副本更适合分析查询。这种日志复制对在领导节点上运行的事务查询产生的开销很小。此外，这种复制的延迟非常短，可以保证 OLAP 的数据新鲜度。我们使用不同的数据副本来分别处理 OLAP 和 OLTP 请求，以避免它们之间的干扰。我们还可以基于行格式和列格式的数据副本来优化 HTAP 请求。TiDB 基于 Raft 协议，提供高可用性、可扩展性和数据一致性。

TiDB 提出了一种创新的解决方案，帮助基于共识算法的 NewSQL 系统演进为 HTAP 系统。New-SQL 系统通过复制 Google Spanner 和 CockroachDB 等数据库，确保 OLTP 请求的高可用性、可扩展性和数据持久性。它们通过通常来自共识算法的复制机制跨数据副本同步数据。基于日志复制，NewSQL 系统可以提供专用于 OLAP 请求的列式副本，从而可以像 TiDB 一样隔离支持 HTAP 请求。

我们的贡献总结如下。

- 我们建议构建一个基于共识算法的 HTAP 系统，并实现了一个基于 Raft 的 HTAP 数据库 TiDB。它是一个开源项目 [7]，为 HTAP 工作负载提供高可用性、一致性、可扩展性、数据新鲜度和隔离性。
- 我们将学习者角色引入 Raft 算法中，为实时 OLAP 查询生成列式存储。
- 我们实现了多 Raft 存储系统并优化其读写，以便系统在扩展到更多节点时提供高性能。
- 我们为大规模 HTAP 查询定制了 SQL 引擎。引擎可以最佳地选择使用基于行的存储和列存储。
- 我们使用 HTAP 基准测试 CH-benCHmark 进行了全面的实验来评估 TiDB 在 OLTP、OLAP 和 HTAP 方面的性能。

本文的其余部分组织如下。我们在第 2 节中描述了基于 Raft 的 HTAP 的主要思想，并在第 3 节中说明了 TiDB 的架构。第 4 节和第 5 节详细阐述了 TiDB 的多 Raft 存储和 HTAP 引擎。第 6 节介绍了实验评估。我们在第 7 节中总结了相关工作。最后，我们在第 8 节中总结了我们的论文。

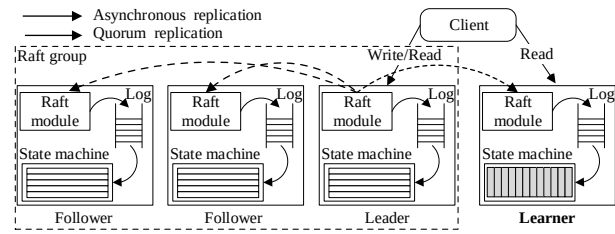


图 1：将柱状学习器添加到 Raft 组

2. 基于 RAFT 的 HTAP

Raft、Paxos 等共识算法是构建一致、可扩展、高可用的分布式系统的基础。它们的优势在于使用复制状态机在服务器之间实时可靠地复制数据。我们调整此功能以将数据复制到不同的服务器以适应不同的 HTAP 工作负载。通过这种方式，我们保证 OLTP 和 OLAP 工作负载相互隔离，而且 OLAP 请求具有新鲜且一致的数据视图。据我们所知，之前还没有使用这些共识算法来构建 HTAP 数据库的工作。

由于 Raft 算法的设计易于理解和实现，因此我们将 Raft 扩展重点放在实现生产就绪的 HTAP 数据库上。如图 1 所示，从高层次来看，我们的想法如下：数据使用行格式存储在多个 Raft 组中以服务事务查询。每个小组由领导者和追随者组成。我们为每个组添加一个学习者角色，以 *asynchronously* 复制领导者的数据。这种方法开销低并保持数据一致性。复制给学习者的数据将转换为基于列的格式。查询优化器经过扩展，可以探索访问基于行和基于列的副本的物理计划。

在标准的 Raft 组中，每个 follower 都可以成为 leader 来服务读写请求。因此，简单地添加更多关注者并不会隔离资源。此外，添加更多的追随者会影响组的性能，因为领导者在响应客户端之前必须等待更多法定节点的响应。因此，我们在 Raft 共识算法中引入了学习者角色。学习者不参与领导者选举，也不是日志复制法定人数的一部分。从领导者到学习者的日志复制是异步的；领导者不需要等待成功才回复客户。领导者和学习者之间的强一致性是在阅读期间强制执行的。根据设计，领导者和学习者之间的日志复制延迟很低，如评估部分所示。

事务查询需要高效的数据更新，而诸如连接或聚合之类的分析查询需要读取列的子集，但需要读取这些列的大量行。基于行的格式可以利用索引来有效地服务事务查询。基于列的格式可以有效地利用数据压缩和矢量化处理。因此，当复制到 Raft 学习器时，数据会从基于行的格式转换为基于列的格式。此外，学习者可以部署在单独的物理资源中。因此，事务查询和分析查询是在隔离的资源中处理的。

我们的设计还提供了新的优化机会。由于数据在基于行的格式和基于列的格式之间保持一致，因此我们的查询优化器可以生成访问其中一个或两个存储的物理计划。

我们提出了扩展 Raft 以满足 HTAP 数据库的新鲜度和隔离性要求的想法。为了让 HTAP 数据库做好生产准备，我们克服了许多工程挑战，主要包括：

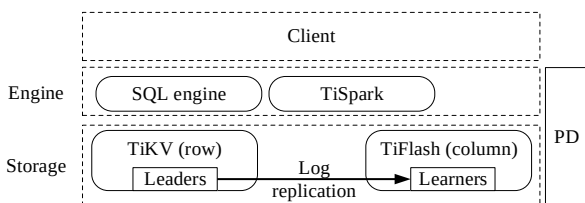


图 2: TiDB 架构

(1) 如何构建一个可扩展的 Raft 存储系统来支持高并发读/写？如果数据量超过 Raft 算法管理的每个节点上的可用空间，我们需要一种分区策略来将数据分布在服务器上。此外，在 Raft 的基本流程中，请求是顺序处理的，任何请求都必须经过 Raft 节点的法定人数批准后才能响应客户端。这个过程涉及网络和磁盘操作，比较耗时。这种开销使得领导者成为处理请求的瓶颈，尤其是在大型数据集上

(2) 如何低延迟地将日志同步到学习器中以保持数据新鲜？正在进行的事务可能会生成一些非常大的日志。这些日志需要在学习者中快速重播和具体化，以便可以读取新数据。将日志数据转换为列格式可能会因架构不匹配而遇到错误。这可能会延迟日志同步。

(3) 如何在保证性能的情况下高效处理事务查询和分析查询？大型事务查询需要读取和写入分布在多个服务器中的大量数据。分析查询也会消耗大量资源，并且不应影响在线交易。为了减少执行开销，他们还需要在行格式存储和列格式存储上选择最佳计划。

在接下来的章节中，我们将详细阐述 TiDB 的设计和实现，以应对这些挑战。

3. 架构

在本节中，我们将描述 TiDB 的高级结构，如图 2 所示。TiDB 支持 MySQL 协议，并且可由 MySQL 兼容的客户端访问。它具有三个核心组件：分布式存储层、放置驱动程序 (PD) 和计算引擎层。

分布式存储层由行存储 (TiKV) 和列存储 (TiFlash) 组成。从逻辑上讲，TiKV 中存储的数据是一个有序的键值映射。每个元组都映射为一个键值对。键由表 ID 和行 ID 组成，值是实际的行数据，其中表 ID 和行 ID 是唯一整数，行 ID 来自主键列。例如，具有四列的元组编码为：

Key: {table{tableID}_record{rowID}}

Value: {col0, col1, col2, col3}

为了横向扩展，我们采用范围分区策略将大型键值映射分割为许多连续的范围，每个范围称为 Region。每个区域都有多个副本以实现高可用性。Raft 共识算法用于维护每个 Region 的副本之间的一致性，形成 Raft 组。不同 Raft 组的领导者将数据从 TiKV 异步复制到 TiFlash。TiKV 和 TiFlash 可以部署在单独的物理资源中，从而在处理事务和分析查询时提供隔离。

Placement Driver (PD) 负责管理区域，包括提供每个密钥的区域和物理位置，以及自动移动区域以平衡工作负载。PD 也是我们的

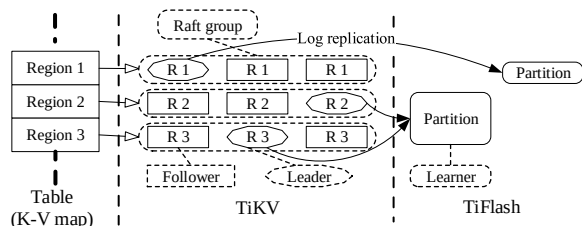


图 3: 多 Raft 存储架构

时间戳预言机，提供严格递增且全局唯一的时间戳。这些时间戳也充当我们的交易 ID。为了稳健性和性能，PD 可能包含多个 PD 成员。PD 没有持久状态，启动时 PD 成员会从其他成员和 TiKV 节点收集所有必要的数据库。

计算引擎层是无状态的并且是可扩展的。我们定制的 SQL 引擎具有基于成本的查询优化器和分布式查询执行器。TiDB 基于 Percolator [33] 实现了两阶段提交 (2PC) 协议来支持事务处理。查询优化器可以根据查询最优选择从 TiKV 和 TiFlash 中读取。

TiDB 的架构满足 HTAP 数据库的要求。TiDB 的每个组件都被设计为具有高可用性和可扩展性。存储层采用 Raft 算法实现数据副本之间的一致性。TiKV 和 TiFlash 之间的低延迟复制使新数据可用于分析查询。查询优化器加上 TiKV 和 TiFlash 之间的强一致性数据，提供快速的分析查询处理，对事务处理影响很小。

除了上述组件之外，TiDB 还集成了 Spark，这有助于集成存储在 TiDB 和 Hadoop 分布式文件系统 (HDFS) 中的数据。TiDB 拥有丰富的生态系统工具，可以将数据导入 TiDB、从 TiDB 导出数据，以及将数据从其他数据库迁移到 TiDB。

在接下来的部分中，我们将深入探讨分布式存储层、SQL 引擎和 TiSpark，以演示 TiDB (一个生产就绪的 HTAP 数据库) 的功能。

4. 多筏存储

图 3 展示了 TiDB 中分布式存储层的架构，其中具有相同形状的对象扮演着相同的角色。存储层由基于行的存储 TiKV 和基于列的存储 TiFlash 组成。存储将一个表映射成一个大的键值映射，该映射被分成许多存储在 TiKV 中的 Region。每个 Region 都使用 Raft 共识算法来维护副本之间的一致性，以实现高可用性。当数据复制到 TiFlash 时，可以将多个 Region 合并为一个分区，以方便表扫描。TiKV 和 TiFlash 之间的数据通过异步日志复制保持一致。由于多个 Raft 组在分布式存储层管理数据，因此我们将其称为多 Raft 存储。在以下几节中，我们详细描述 TiKV 和 TiFlash，重点关注使 TiDB 成为生产就绪的 HTAP 数据库的优化。

4.1 基于行的存储 (TiKV)

TiKV 部署由许多 TiKV 服务器组成。使用 Raft 在 TiKV 服务器之间复制区域。每个 TiKV 服务器可以是不同 Region 的 Raft 领导者或跟随者。在每个 TiKV 服务器上，数据和元数据都保存到 RocksDB，这是一个可嵌入、持久的键值存储 [5]。每个区域都有一个可配置的最大大小，默认为 96 MB。Raft Leader 的 TiKV 服务器处理相应 Region 的读/写请求。

当 Raft 算法响应读写请求时，Leader 和 Follower 之间会执行基本的 Raft 流程：

(1) Region Leader收到来自SQL引擎层的请求。(2)领导者将请求附加到其日志中。(3)领导者将新的日志条目发送给其追随者，追随者又将这些条目追加到其日志中。(4)领导者等待其追随者响应。如果法定数量的节点响应成功，则领导者提交请求并在本地应用它。(5)领导者将结果发送给客户端并继续处理传入的请求。

这个过程保证了数据的一致性和高可用性。然而，它并不能提供高效的性能，因为这些步骤是连续发生的，并且可能会产生大量的 I/O 开销（磁盘和网络）。以下各节描述了如何优化此过程以实现高读/写吞吐量，即解决第 2 节中描述的第一个挑战。

4.1.1 Optimization between Leaders and Followers

在上述过程中，第二步和第三步可以并行发生，因为它们之间不存在依赖关系。因此，领导者在本地追加日志，同时将日志发送给跟随者。如果在领导者上追加日志失败，但一定数量的追随者成功追加日志，则仍然可以提交日志。第三步，当leader向follower发送日志时，leader会缓冲日志条目并将其批量发送给其follower。发送日志后，领导者不必等待追随者响应。相反，它可以假设成功并发送带有预测日志索引的进一步日志。如果发生错误，领导者会调整日志索引并重新发送复制请求。在第四步中，应用已提交日志条目的领导者可以由不同的线程异步处理，因为在此阶段不存在一致性风险。基于上述优化，Raft流程更新如下：

(1) Leader接收来自SQL引擎层的请求。(2)leader向follower发送相应的日志，并并行在本地追加日志。(3)领导者继续接收客户端的请求并重复步骤(2)。(4)leader提交日志并发送给另一个线程应用。(5)领导者应用日志后，将结果返回给客户端。

在这个优化过程中，来自客户端的任何请求仍然会运行所有 Raft 步骤，但来自多个客户端的请求会并行运行，因此整体吞吐量会增加。

4.1.2 Accelerating Read Requests from Clients

从 TiKV Leader 读取数据具有线性化语义。这意味着当在时间 t 从区域领导者读取值时，领导者不得为 t 之后的读取请求返回该值的先前版本。这可以通过使用 Raft 来实现，如上所述：为每个读取请求发出一个日志条目，并在返回之前等待该条目被提交。然而，这个过程的成本很高，因为日志必须在 Raft 组中的大多数节点之间复制，从而产生网络 I/O 的开销。为了提高性能，我们可以避免日志同步阶段。

Raft 保证一旦领导者成功写入数据，领导者就可以响应任何读取请求，而无需跨服务器同步日志。然而，在领导者选举之后，领导者角色可能会在 Raft 组中的服务器之间移动。为了实现领导者的读取，TiKV 实现了以下读取优化，如[29]中所述。

第一种方法称为 *read index*。当领导者响应读请求时，它将当前的提交索引记录为本地读索引，然后向追随者发送心跳消息以确认其领导者角色。如果它确实是领导者，那么一旦它的应用索引大于或等于读取索引，它就可以返回该值。这种方法提高了读取性能，但会带来一点网络开销。

另一种方法是 *lease read*，它减少了读取索引引起的心跳网络开销。Leader和Follower商定一个租约期限，在此期间Follower不会发出选举请求，这样Leader就不会被更换。在租用期间，领导者可以响应任何读取请求，而无需连接到其追随者。如果每个节点上的 CPU 时钟相差不大，则此方法效果很好。

除了领导者之外，追随者也可以响应客户端的读取请求，这称为 *follower read*。follower收到读请求后，会向leader询问最新的读索引。如果本地应用的索引等于或大于读取的索引，则follower可以将该值返回给客户端；否则，必须等待日志被应用。Follower读可以减轻热点Region的leader的压力，从而提高读性能。然后通过添加更多关注者来进一步提高读取性能。

4.1.3 Managing Massive Regions

海量区域分布在服务器集群上。服务器和数据大小是动态变化的，Region可能会在某些服务器上聚集，特别是领导副本。这会导致某些服务器的磁盘被过度使用，而其他服务器的磁盘则空闲。此外，服务器可以添加到集群或从集群中移出。

为了平衡服务器之间的区域，放置驱动程序 (PD) 在调度区域时会限制副本的数量和位置。一个关键限制是在不同的 TiKV 实例上放置至少三个 Region 副本以确保高可用性。PD是通过心跳从服务器收集特定信息来初始化的。它还监控每个服务器的工作负载，并将热点区域迁移到不同的服务器，而不影响应用程序。

另一方面，维护大量区域涉及发送心跳和管理元数据，这可能会导致大量网络 and 存储开销。但是，如果Raft组没有任何工作负载，则不需要心跳。根据Region的工作负载繁忙程度，我们可以调整发送心跳的频率。这减少了遇到网络延迟或节点过载等问题的可能性。

4.1.4 Dynamic Region Split and Merge

大区域可能会变得太热而无法在合理的时间内读取或写入。热点或大型区域应分为较小的区域，以更好地分配工作负载。另一方面，可能很多Region都很小，很少被访问；然而，系统仍然需要维护心跳和元数据。在某些情况下，维护这些小区域会产生大量网络和 CPU 开销。因此，有必要合并更小的Region。请注意，为了保持 Region 之间的顺序，我们仅合并键空间中相邻的 Region。根据观察到的工作负载，PD 动态向 TiKV 发送拆分和合并命令。

分割操作将一个 Region 分成几个新的、更小的 Region，每个 Region 覆盖原始 Region 中的连续范围的键。覆盖最右边范围的Region复用了原Region的Raft组。其他区域使用新的 Raft 组。split过程与Raft过程中的普通更新请求类似：

表1: 日志回放和解码

Raw logs	<pre>{1}{insert}{prewritten@1}{k1→(a1,b1)} {2}{insert}{prewritten@2}{k2→(a2,b2)} {3}{update}{prewritten@3}{k3→(a3,b3)} {1}{insert}{rollbacked@1} {2}{insert}{committed#4} {3}{update}{committed#5} {4}{delete}{prewritten@6}{k4} {4}{delete}{committed#7}</pre>
Compacted logs	<pre>{2}{insert}{prewritten@2}{k2→(a2,b2)} {3}{update}{prewritten@3}{k3→(a3,b3)} {2}{insert}{committed#4} {3}{update}{committed#5} {4}{delete}{prewritten@6}{k4} {4}{delete}{committed#7}</pre>
Decoded tuples	<pre>{insert}{#4}{k2→(a2,b2)} {update}{#5}{k3→(a3,b3)} {delete}{#7}{k4}</pre>
Columnar data	<pre>{insert,update,delete,} {#4,#5,#7,} {k2,k3,k4,} {a2,a3,,} {b2,b3,,}</pre>

- (1) PD 向 Region 的领导者发出分裂命令。
- (2) Leader 收到 split 命令后，将命令转化为日志，并将日志复制到其所有 follower 节点。该日志仅包含分割命令，而不是修改实际数据。
- (3) 一旦仲裁复制了日志，Leader 就会提交 split 命令，该命令将应用到 Raft 组中的所有节点。应用过程涉及更新原始区域的范围和纪元元数据，以及创建新区域以覆盖剩余范围。请注意，该命令以原子方式应用并同步到磁盘。
- (4) 对于分割 Region 的每个副本，都会创建一个 Raft 状态机并开始工作，形成一个新的 Raft 组。原 Region 的 Leader 将分裂结果报告给 PD。分割过程完成。

请注意，当大多数节点提交拆分日志时，拆分过程就会成功。与提交其他 Raft 日志类似，而不是要求所有节点完成 Region 分裂。分裂后，如果网络被分区，则具有最新纪元的节点组获胜。区域分割的开销很低，因为只需要元数据更改。分割命令完成后，由于 PD 的定期负载均衡，新分割的 Region 可能会跨服务器移动。

合并两个相邻区域与拆分一个区域相反。PD 移动两个区域的副本，将它们放置在不同的服务器上。然后，通过两阶段操作在每个服务器上本地合并两个 Region 的共置副本；即停止一个 Region 的服务并与另一个 Region 合并。这种方法与分割 Region 不同，因为它不能使用两个 Raft 组之间的日志复制过程来同意合并它们。

4.2 列式存储 (TiFlash)

尽管我们如上所述优化了从 TiKV 读取数据，但 TiKV 中的行格式数据不太适合快速分析。因此，我们将列存储 (TiFlash) 合并到 TiDB 中。TiFlash 由学习器节点组成，学习器节点仅接收来自 Raft 组的 Raft 日志，并将行格式元组转换为列式数据。他们不参与 Raft 协议来提交日志或选举领导者，因此他们在 TiKV 上产生的开销很小。

用户可以使用 SQL 语句为表设置列格式副本：

```
ALTER TABLE x SET TiFLASH REPLICA n;
```

其中 x 是表的名称， n 是副本数。默认值为 1。添加列副本类似于向表添加异步索引。TiFlash 中的每个表都被分为许多分区，每个分区都覆盖了一系列连续的元组，与 TiKV 中的几个连续的 Region 一致。较大的分区有利于范围扫描。

当初始化 TiFlash 实例时，相关区域的 Raft 领导者开始将其数据复制给新的学习者。如果数据太多而无法快速同步，领导者会发送其数据的快照。初始化完成后，TiFlash 实例开始侦听来自 Raft 组的更新。学习者节点收到日志包后，将日志应用到本地状态机，包括重放日志、转换数据格式以及更新本地存储中的引用值。

在以下部分中，我们将说明 TiFlash 如何有效地应用日志并与 TiKV 保持一致的视图。这满足了我们在第 2 节中描述的第二个挑战。

4.2.1 Log Replayer

根据 Raft 算法，学习器节点接收到的日志是可线性化的。为了保持提交数据的线性化语义，它们根据先进先出 (FIFO) 策略进行重放。日志重放分为三个步骤：

(1) 压缩日志：根据后面 5.1 节描述的事务模型，事务日志分为三种状态：预写、已提交或已回滚。回滚日志中的数据不需要写入磁盘，因此紧凑进程根据回滚日志删除预先写入的无效日志，并将有效日志放入缓冲区。

(2) 解码元组：缓冲区中的日志被解码为行格式元组，去除有关事务的冗余信息。然后，解码后的元组被放入行缓冲区中。

(3) 转换数据格式：如果行缓冲区中的数据大小超过大小限制或其持续时间超过时间间隔限制，则这些行格式元组将转换为列式数据并写入本地分区数据池。转换指的是本地缓存的 schema，它会定期与 TiKV 同步，如下所述。

为了说明日志重放过程的细节，请考虑以下示例。我们将每个 Raft 日志项抽象为事务 ID-操作类型[事务状态][@start ts][#commit ts] 操作数据。根据典型的 DML，操作类型包括插入、更新和删除元组。事务状态可以是预先写入的、已提交的或回滚的。操作数据可以是特定插入或更新的元组，或者删除的键。

在表 1 所示的示例中，原始日志包含八项尝试插入两个元组、更新一个元组和删除一个元组。但是插入 k1 会回滚，因此仅保留 8 个原始日志项中的 6 个，从中解码出三个元组。最后，三个解码的元组被转换为五列：操作类型、提交时间戳、键和两列数据。这些列被附加到 DeltaTree。

4.2.2 Schema Synchronization

为了实时将元组转换为柱状格式，学习者节点必须了解最新的模式。这种有模式的过程与 TiKV 上的无模式操作不同，后者将元组编码为字节数组。最新的 schema 信息存储在 TiKV 中。为了减少 TiFlash 向 TiKV 请求最新模式的次数，每个学习者节点都维护一个模式缓存。

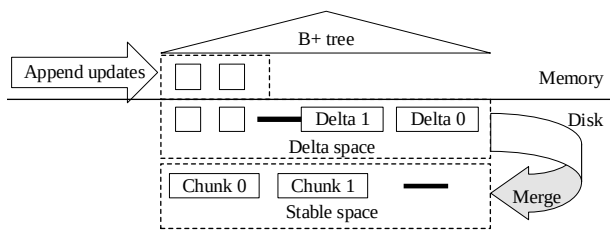


图 4：柱状 Delta 树

缓存通过 schema 同步器与 TiKV 的 schema 同步。如果缓存的模式已过期，则正在解码的数据与本地模式之间存在不匹配，并且必须重新转换数据。模式同步的频率和模式不匹配的数量之间存在权衡。我们采取两阶段策略：

- 定期同步：模式同步器定期从 TiKV 获取最新模式并将更改应用于其本地缓存。在大多数情况下，这种临时同步会降低同步模式的频率。
- 强制同步：如果 schema 同步器检测到不匹配的 schema，它会主动从 TiKV 获取最新的 schema。当元组和模式之间的列号不同或列值溢出时，可以触发此操作。

4.2.3 Columnar Delta Tree

为了以高吞吐量高效地写入和读取列式数据，我们设计了一种新的列式存储引擎 DeltaTree，它立即附加增量更新，然后将它们与每个分区之前的稳定版本合并。增量更新和稳定数据分别存储在 DeltaTree 中，如图 4 所示。在稳定空间中，分区数据以块的形式存储，每个块覆盖分区元组的较小范围。此外，这些行格式元组是逐列存储的。相比之下，增量按照 TiKV 生成它们的顺序直接附加到增量空间中。TiFlash 中列式数据的存储格式类似于 Parquet [4]。它还将行组存储到柱状块中。不同的是，TiFlash 将行组的列数据及其元数据存储到不同的文件中以并发更新文件，而不是 Parquet 中仅存储一个文件。TiFlash 仅使用常见的 LZ4 [2] 压缩来压缩数据文件以节省磁盘大小。

新传入的增量是插入数据或删除范围的原子批次。这些增量被缓存到内存中并具体化到磁盘中。它们按顺序存储，因此实现了预写日志（WAL）的功能。这些增量通常存储在许多小文件中，因此在读取时会产生大量 IO 开销。为了降低成本，我们定期将这些小增量压缩为一个较大的增量，然后将较大的增量刷新到磁盘并替换之前具体化的增量。传入增量的内存中副本有助于读取最新数据，如果旧增量达到有限大小，则会将其删除。

当读取某些特定元组的最新数据时，有必要将所有增量文件与其稳定元组合并（即读取放大），因为事先不知道相关增量分布在哪里。由于读取大量文件，这样的过程是昂贵的。此外，许多增量文件可能包含无用的数据（即空间放大），这些数据浪费存储空间并减慢它们与稳定元组的合并速度。因此，我们定期将增量合并到稳定空间中。每个增量文件及其相关块都被读入内存并合并。增量中插入的元组被添加到稳定的元组中，修改的元组替换原始元组，删除的元组被移动。合并的块自动替换磁盘中的原始块。

表2：DeltaTree和LSM树的读取性能

Tuple number	Storage engine	Transactions per second			
		150	1300	8000	14000
100 M	Delta tree	0.49	0.48	0.5	0.45
	LSM tree	1.01	0.97	0.94	0.95
200 M	Delta tree	0.71	0.74	0.72	0.75
	LSM tree	1.59	1.63	1.67	1.64

合并增量的成本很高，因为相关的键在增量空间中是无序的。这种无序还会减慢将增量与稳定块集成以返回读取请求的最新数据的速度。因此，我们在delta空间的顶部构建一个B+树索引。每个更新的增量项都会插入按其键和时间戳排序的B+树中。此顺序优先级有助于在响应读取请求时有效地定位一系列键的更新或在增量空间中查找单个键。此外，B+树中的有序数据很容易与稳定块合并。

我们进行了一个微观实验，将 DeltaTree 的性能与 TiFlash 中的日志结构合并（LSM）树 [28] 进行比较，其中数据根据 Raft 日志更新时读取。我们设置了 3 个 TiKV 节点和 1 个 TiFlash 节点，硬件配置列于实验部分。我们在 TiKV 上运行 Sysbench [6] 的唯一写入工作负载，并在 TiFlash 上运行 “`select count(id), count(k) from sbtest1`”。为了避免数据压缩的大量写入放大，我们使用通用压缩而不是级别样式压缩来实现 LSM 存储引擎。ClickHouse [1]（一个面向列的 OLAP 数据库）也采用了这种实现。

如表 2 所示，无论有 100 个还是 2 亿个元组，以及事务工作负载，从 delta 树读取的速度大约是 LSM 树的两倍。这是因为在增量树中，每次读取最多访问在 B+ 树中索引的一级增量文件，而在 LSM 树中它访问更多重叠文件。在不同的写入工作负载下，性能几乎保持稳定，因为增量文件的比例几乎相同。虽然 DeltaTree (16.11) 的写放大大于 LSM 树 (4.74)，但是可以接受的。

4.2.4 Read Process

与跟随者读取一样，学习者节点提供快照隔离，因此我们可以在特定时间戳从 TiFlash 读取数据。收到读取请求后，学习器向其领导者发送读取索引请求，以获取涵盖所请求时间戳的最新数据。作为响应，领导者将引用的日志发送给学习者，学习者重播并存储日志。一旦日志写入 DeltaTree，就会读取 DeltaTree 中的特定数据以响应读取请求。

5.HTAP引擎

为了解决第2节中提到的第三个挑战，即处理大规模事务和分析查询，我们提供了一个SQL引擎来评估事务和分析查询。SQL引擎采用Percolator模型来实现分布式集群中的乐观锁和悲观锁。SQL引擎通过使用基于规则和成本的优化器、索引并将计算下推到存储层来加速分析查询。我们还实现了TiSpark来连接Hadoop生态系统并增强OLAP能力。HTAP请求可以在隔离的存储和引擎服务器中单独处理。特别是，SQL引擎和TiSpark受益于同时使用行存储和列存储以获得最佳结果。

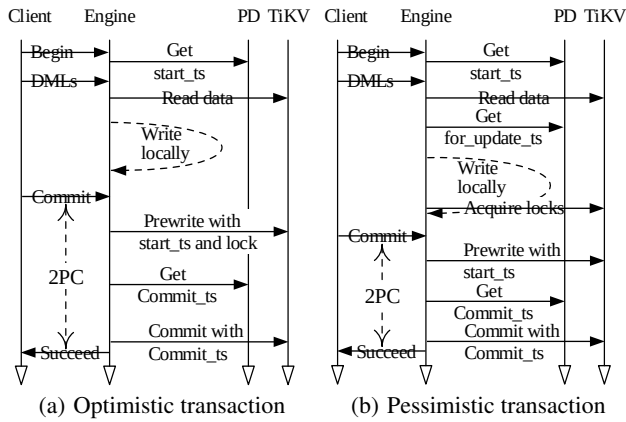


图5: 乐观和悲观交易流程

5.1 事务处理

TiDB 为 ACID 事务提供快照隔离 (SI) 或可重复读 (RR) 语义。SI 允许事务中的每个请求读取一致版本的数据。RR 意味着事务中的不同语句可能会读取同一键的不同值，但重复读取（即具有相同时间戳的两次读取）将始终读取相同的值。我们的实现基于多版本并发控制 (MVCC)，避免读写锁定并防止写入冲突。

在 TiDB 中，事务是 SQL 引擎、TiKV 和 PD 之间协作的。事务期间每个组件的职责如下：

- SQL引擎：协调事务。它接收来自客户端的写入和读取请求，将数据转换为键值格式，并使用两阶段提交（2PC）将事务写入TiKV。
- PD：管理逻辑Region和物理位置；提供全局、严格递增的时间戳。
- TiKV：提供分布式事务接口，实现MVCC，将数据持久化到磁盘。

TiDB 实现了乐观锁和悲观锁。它们改编自Percolator模型，选择一个键作为主键并用它来代表交易的状态，并基于2PC来进行交易。乐观事务的过程如图 5 左侧所示。（为简单起见，该图忽略了异常处理。）

(1) 在收到客户端的“开始”命令后，SQL 引擎向 PD 请求时间戳以用作事务的开始时间戳 (*start_ts*)。 (2) SQL 引擎通过从 TiKV 读取数据并写入本地内存来执行 SQL DML。 TiKV 提供的数据具有事务 *start_ts* 之前的最新提交时间戳 (*commit_ts*)。 (3) 当SQL引擎收到客户端的提交命令时，启动2PC协议。它随机选择一个主键，并行锁定所有键，并将预写入发送到 TiKV 节点。 (4) 如果所有预写成功，SQL 引擎向 PD 请求事务 *commit_ts* 的时间戳，并向 TiKV 发送提交命令。 TiKV 提交主键并向 SQL 引擎发送成功响应。 (5) SQL引擎向客户端返回成功。 (6) SQL引擎通过向TiKV发送进一步的提交命令来异步并行地提交辅助键并清除锁。

乐观事务和悲观事务之间的主要区别在于何时获取锁。在乐观事务中，锁是在预写阶段增量获取的（上面的步骤 3）。在悲观事务中，在预写之前执行 DML 时会获取锁（步骤 2 的一部分）。这意味着一旦预写开始，事务就不会因为与另一个事务冲突而失败。（由于网络分区或其他问题，它仍然可能失败。）

当锁定悲观事务中的键时，SQL 引擎会获取一个新的时间戳，称为 *for_update_ts*。如果 SQL 引擎无法获取锁，它可以从该锁开始重试事务，而不是回滚并重试整个事务。在读取数据时，TiKV 使用 *for_update_ts* 而不是 *start_ts* 来决定可以读取 key 的哪些值。通过这种方式，悲观事务可以维持 RR 隔离级别，即使部分重试事务也是如此。

对于悲观事务，用户还可以选择仅需要已提交读 (RC) 隔离级别。这会减少事务之间的冲突，从而提高性能，但代价是减少孤立的事务。实现上的差异在于，对于 RR，如果读取尝试访问被另一个事务锁定的密钥，TiKV 必须报告冲突；对于 RC，读时可以忽略锁。

TiDB 在没有集中锁管理器的情况下实现分布式事务。锁存储在 TiKV 中，具有很高的可扩展性和可用性。此外，SQL 引擎和 PD 服务器可扩展以处理 OLTP 请求。跨服务器同时运行多个事务可实现高度并行性。

时间戳是向 PD 请求的。每个时间戳包括物理时间和逻辑时间。物理时间是指当前时间，精度为毫秒，逻辑时间占用 18 位。因此理论上 PD 每毫秒可以分配 2^{18} 个时间戳。实际上，它每秒可以生成大约 100 万个时间戳，因为分配时间戳只需要几个周期。客户端每批请求一次时间戳以分摊开销，特别是网络延迟。目前，在我们的实验和许多生产环境中，获取时间戳并不是性能瓶颈。

5.2 分析处理

在本节中，我们将描述针对 OLAP 查询的优化，包括优化器、索引以及我们定制的 SQL 引擎和 TiSpark 中的下推计算。

5.2.1 Query Optimization in SQL Engine

TiDB 实现的查询优化器具有两个阶段的查询优化：基于规则的查询优化 (RBO)，生成逻辑计划，然后基于成本的优化 (CBO)，将逻辑计划转换为物理计划。我们的 RBO 有一套丰富的转换规则，包括裁剪不需要的列、消除投影、下推谓词、派生谓词、常量折叠、消除“group by”或外连接以及取消嵌套子查询。我们的 CBO 根据执行成本从候选计划中选择最便宜的计划。请注意，TiDB 提供了 TiKV 和 TiFlash 两种数据存储，因此扫描表通常有三种选择：扫描 TiKV 中的行格式表、扫描 TiKV 中带索引的表以及扫描 TiFlash 中的列。

索引对于提高数据库的查询性能非常重要，索引通常用于点获取或范围查询，为散列连接和合并连接提供更便宜的数据扫描路径。TiDB 实现了可扩展索引以在分布式环境中工作。由于维护索引会消耗大量资源，并且可能会影响在线事务和分析，因此我们在后台异步构建或删除索引。索引是

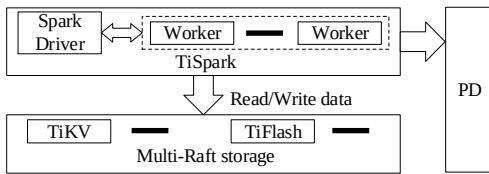


图 6: TiSpark 和 TiDB 的交互

按照与数据相同的方式按 Region 进行拆分，并以键值形式存储在 TiKV 中。唯一键索引上的索引项编码为：

Key: {table{tableID}_index{indexID}_indexedColValue}

Value: {rowID} 非唯一索引上的索引项解码为：

Key: {table{tableID}_index{indexID}_indexedColValue_rowID}

Value: {null}

使用索引需要进行二分搜索来定位包含索引相关部分的区域。为了增加索引选择的稳定性并减少物理优化的开销，我们使用天际线剪枝算法来消除无用的候选索引。如果有多个候选索引匹配不同的查询条件，我们会合并部分结果（即一组合格的行 ID）以获得精确的结果集。

物理计划（CBO 的结果）由 SQL 引擎层使用拉迭代器模型 [17] 执行。通过将一些计算下推到存储层可以进一步优化执行。在存储层中，执行计算的组件称为 *coprocessor*。协处理器在不同服务器上并行执行执行计划的子树。这减少了必须从存储层发送到引擎层的元组数量。例如，通过评估协处理器中的过滤器，被拒绝的元组在存储层中被过滤掉，并且只有被接受的元组需要被发送到引擎层。协处理器可以评估逻辑运算、算术运算和其他常见功能。在某些情况下，它可以执行聚合和 TopN。协处理器可以通过向量化操作进一步提高性能：不是对整行进行迭代，而是对行进行批处理，并按列组织数据，从而实现更高效的迭代。

5.2.2 TiSpark

为了帮助 TiDB 连接 Hadoop 生态，TiDB 在多 Raft 存储上添加了 TiSpark。除了 SQL 之外，TiSpark 还支持机器学习库等强大的计算，并且可以处理来自外部 TiDB 的数据。

图 6 显示了 TiSpark 与 TiDB 的集成方式。在 TiSpark 中，Spark 驱动程序从 TiKV 读取元数据来构建 Spark 目录，包括表模式和索引信息。Spark 驱动程序向 PD 请求时间戳以从 TiKV 读取 MVCC 数据，以确保获得一致的数据库快照。与 SQL 引擎一样，Spark Driver 可以将计算下推到存储层上的协处理器并使用可用索引。这是通过修改 Spark 优化器生成的计划来完成的。我们还定制了一些读取操作来从 TiKV 和 TiFlash 读取数据，并将它们组装成行供 Spark 工作线程使用。例如，TiSpark 可以同时从多个 TiDB Region 读取，并且可以并行地从存储层获取索引数据。为了减少对 Spark 特定版本的依赖，大多数这些功能都在附加包中实现。

TiSpark 与普通连接器有两个不同之处。不仅可以同时读取多个数据 Region，还可以并行从存储层获取索引数据。读取索引可以方便 Spark 中的优化器选择最优计划来降低执行成本。另一方面，TiSpark 修改计划

从 Spark 中的原始优化器生成，将部分执行下推到存储层中的协处理器，这进一步减少了执行开销。除了从存储层读取数据之外，TiSpark 还支持通过事务加载存储层的大数据。为了实现这一点，TiSpark 采用两阶段提交和锁定义表。

5.3 隔离与协调

资源隔离是保证事务查询性能的有效方法。分析查询通常会消耗大量资源，例如 CPU、内存和 I/O 带宽。如果这些查询与事务性查询一起运行，则后者可能会严重延迟。这一原则已在之前的工作中得到验证 [24, 34]。为了避免 TiDB 中的这个问题，我们将分析查询和事务查询安排在不同的引擎服务器上，并将 TiKV 和 TiFlash 部署在不同的服务器上。事务性查询主要访问 TiKV，而分析查询主要访问 TiFlash。通过 Raft 维护 TiKV 和 TiFlash 之间的数据一致性的开销很低，因此使用 TiFlash 运行分析查询对事务处理的性能影响很小。

TiKV 和 TiFlash 之间的数据是一致的，因此可以通过从 TiKV 或 TiFlash 读取来服务查询。因此，我们的查询优化器可以从更大的物理计划空间中进行选择，并且最佳计划可以从 TiKV 和 TiFlash 中读取。TiKV 访问表时提供行扫描和索引扫描，TiFlash 支持列扫描。

这三种访问路径在执行成本和数据顺序属性方面彼此不同。行扫描和列扫描提供按主键排序；索引扫描根据键的编码提供了多种排序。不同路径的成本取决于平均元组/列/索引大小 ($S_{tuple/col/index}$) 和元组/区域的估计数量 ($N_{tuple/reg}$)。我们将数据扫描的 I/O 开销表示为 f_{scan} ，将文件查找成本表示为 f_{seek} 。查询优化器根据式 (1) 选择最优访问路径。如式

(2) 所示，行扫描的成本来自于扫描连续行数据和寻找 Region 文件。列扫描的成本（等式 (3)）是扫描 m 列的总和。如果索引列不满足表扫描所需的列，则索引扫描（式 (4)）应考虑扫描索引文件的成本和扫描数据文件的成本（即双读）。请注意，双重读取通常随机扫描元组，这涉及在等式 (5) 中寻找更多文件。

$$C_{opt.scan} = \min(C_{col.scan}, C_{row.scan}, C_{index.scan}) \quad (1)$$

$$C_{row.scan} = S_{tuple} \cdot N_{tuple} \cdot f_{scan} + N_{reg} \cdot f_{seek} \quad (2)$$

$$C_{col.scan} = \sum_{j=1}^m (S_{col.j} \cdot N_{tuple} \cdot f_{scan} + N_{reg.j} \cdot f_{seek}) \quad (3)$$

$$C_{index.scan} = S_{index} \cdot N_{tuple} \cdot f_{scan} + N_{reg} \cdot f_{seek} + C_{double.read} \quad (4)$$

$$C_{double.read} = \begin{cases} 0 & (\text{if without double read}) \\ S_{tuple} \cdot N_{tuple} \cdot f_{scan} + N_{tuple} \cdot f_{seek} & \end{cases} \quad (5)$$

作为查询优化器何时选择行格式存储和列格式存储来访问同一查询中的不同表的示例，请考虑

“select T.a, S.a from T join S on T.b=S.b

where T.a between 1 and 100”。这是一个典型的连接查询，其中 T 和 S 在行存储中的 a 列上有索引，以及列副本。最好使用索引从行存储访问 T，从列存储访问 S。这是因为查询需要 T 中的一系列完整元组，并且通过索引按元组访问数据比列存储便宜。另一方面，使用列存储时，获取 S 的两个完整列会更便宜。

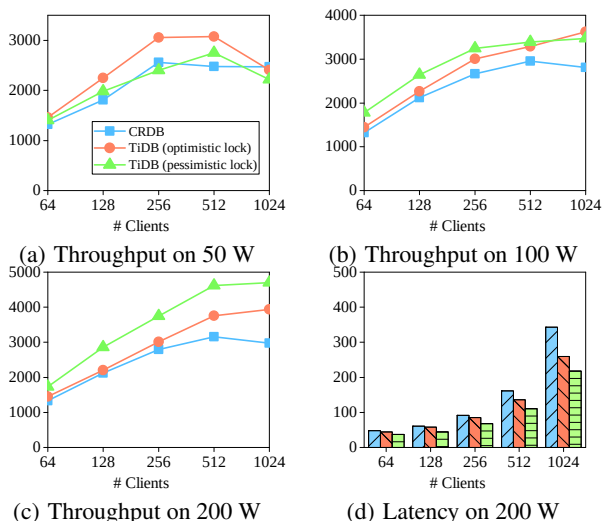


图 7: OLTP 性能

TiKV 和 TiFlash 配合仍然可以保证隔离性能。对于分析查询，只有小范围扫描或点获取扫描可以通过follower读访问TiKV，这对leader影响不大。我们还将 TiKV 上用于分析查询的默认访问表大小限制为最多 500 MB。事务查询可以从 TiFlash 访问列式数据来检查一些约束，例如唯一性。我们为特定表设置多个列副本，并且一个表副本专用于事务查询。在单独的服务器上处理事务查询可以避免影响分析查询。

6. 实验

本节我们首先分别评估 TiDB 的 OLTP 和 OLAP 能力。对于 OLAP，我们研究了 SQL 引擎选择 TiKV 和 TiFlash 的能力，并将 TiSpark 与其他 OLAP 系统进行比较。然后，我们测量 TiDB 的 HTAP 性能，包括 TiKV 和 TiFlash 之间的日志复制延迟。最后，我们在隔离性方面比较 TiDB 和 MemSQL。

6.1 实验设置

簇。我们在六台服务器的集群上进行全面的实验；每个都有 188 GB 内存和两个 Intel® Xeon® CPU E5-2630 v4 处理器，即两个 NUMA 节点。每个处理器有 10 个物理核心（20 个线程）和 25 MB 共享 L3 缓存。这些服务器运行 Centos 版本 7.6.1810，并通过 10 Gbps 以太网连接。

工作量。我们的实验是使用 CH-benCHmark 在混合 OLTP 和 OLAP 工作负载下进行的。源代码在线发布[7]。该基准测试由标准 OLTP 和 OLAP 基准测试组成：TPC-C 和 TPC-H。它是根据 TPC-C 基准测试的未修改版本构建的。OLAP 部分包含受 TPC-H 启发的 22 个分析查询，其模式从 TPC-H 改编为 CH-benCHmark 模式，以及三个缺失的 TPC-H 关系。在运行时，两个工作负载由多个客户端同时发出；实验中客户端的数量是不同的。吞吐量分别以每秒查询数 (QPS) 或每秒事务数 (TPS) 来衡量。CH-benCHmark 中的数据单位称为仓库，与 TPC-C 相同。100 个仓库大约需要 70 GB 内存。

6.2 OLTP性能

我们在 CH-benCHmark 的 OLTP 部分下通过乐观锁定或悲观锁定来评估 TiDB 的独立 OLTP 性能；即 TPC-C 基准。我们比较 TiDB 的 per-

表3: 时间戳检索的性能

Servers	Per-server statistics			
	Count	Max (ms)	>= 1ms	>=2ms
6	602594	1	248	0
12	319585	2	254	100
24	135278	2	322	15

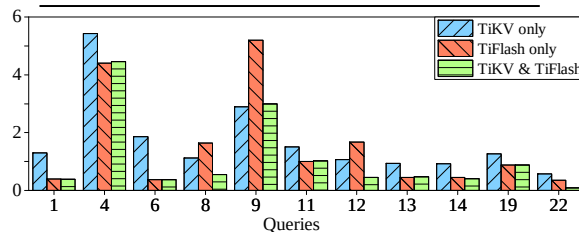


图 8: 选择 TiKV 或 TiFlash 进行分析查询

CockroachDB (CRDB) 是另一个分布式 NewSQL 数据库。CRDB 部署在六台同构服务器上。对于 TiDB，SQL 引擎和 TiKV 部署在六台服务器上，它们的实例分别绑定到每台服务器上的两个 NUMA 节点。PD 部署在六台服务器中的三台上。为了平衡请求，TiDB 和 CRDB 都通过 HAProxy 负载均衡器进行访问。我们使用不同数量的客户端测量 50、100 和 200 个仓库的吞吐量和平均延迟。

图 7(b) 和 7(c) 中的吞吐量图与图 7(a) 不同。在图 7(a) 中，对于少于 256 个客户端，无论是乐观锁还是悲观锁，TiDB 的吞吐量都随着客户端数量的增加而增加。对于超过 256 个客户端，乐观锁的吞吐量保持稳定，然后开始下降，而悲观锁的吞吐量在 512 个客户端时达到最大值然后下降。图 7(b) 和 7(c) 中 TiDB 的吞吐量不断增加。此结果是预期的，因为高并发和小数据量时资源争用最严重。

一般来说，乐观锁的性能优于悲观锁，除非数据量较小且并发性较高（50 或 100 个仓库上有 1024 个客户端），此时资源争用严重并导致重试许多乐观事务。由于 200 个仓库的资源争用较轻，因此乐观锁定仍会产生更好的性能。

在大多数情况下，TiDB 的吞吐量高于 CRDB，特别是在大型仓库上使用乐观锁时。即使采用悲观锁进行公平比较（CRDB 总是使用悲观锁），TiDB 的性能仍然更高。我们认为 TiDB 的性能优势归功于事务处理和 Raft 算法的优化。

图 7(d) 显示更多的客户端会导致更多的延迟，特别是在达到最大吞吐量之后，因为更多的请求必须等待更长的时间。这也是仓库较少时延迟较高的原因。对于某些客户端，更高的吞吐量会导致 TiDB 和 CRDB 的延迟更短。50 个和 100 个仓库也有类似的结果。

我们评估从 PD 请求时间戳的性能，因为这可能是一个潜在的瓶颈。我们使用 1200 个客户端来连续请求时间戳。客户端位于集群中的不同服务器上。每个客户端模拟 TiDB，批量向 PD 发送时间戳请求。如表 3 所示，六台服务器每台每秒可以接收 602594 个时间戳，这是运行 TPC-C 基准测试时所需速率的 100 多倍。运行 TPC-C 时，TiDB 每台服务器每秒最多请求 6000 个时间戳。当增加服务器数量时，每个服务器上接收到的时间戳数量减少，但时间戳总数几乎相同。这

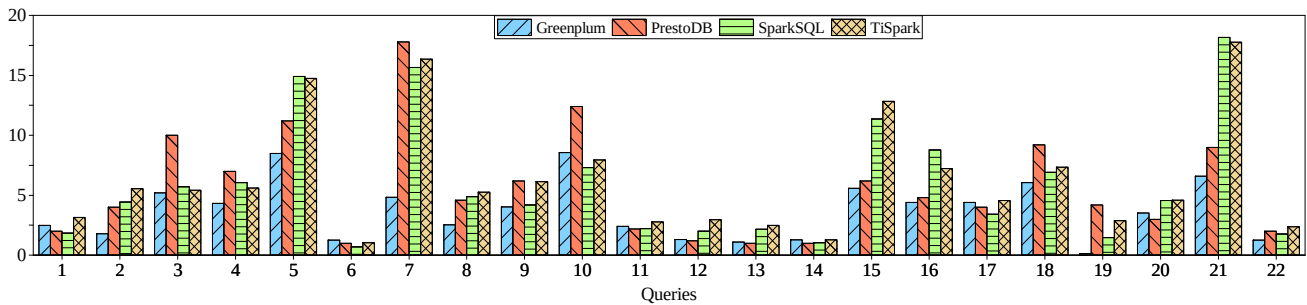


图 9: CH-benCHmark 分析查询的性能比较

速率大大超过任何现实生活的需求。关于延迟，只有一小部分请求花费 1 毫秒或 2 毫秒。我们的结论是，从 PD 获取时间戳目前并不是 TiDB 的性能瓶颈。

6.3 OLAP性能

我们从两个角度评估 TiDB 的 OLAP 性能。首先，我们评估 SQL 引擎在具有 100 个仓库的 CH-benCHmark OLAP 部分下最佳选择行存储或列存储的能力。我们设置了三种类型的存储：仅 TiKV、仅 TiFlash、以及 TiKV 和 TiFlash。我们运行每个查询五次并计算平均执行时间。如图 8 所示，仅从单一类型的存储中获取数据，这两种存储都不优越。从 TiKV 和 TiFlash 请求数据总是表现更好。

Q8、Q12 和 Q22 产生有趣的结果。在 Q8 和 Q12 中，仅 TiKV 的情况比仅 TiFlash 的情况花费的时间更少，但在 Q22 中花费的时间更多。TiKV 和 TiFlash 情况比仅 TiKV 和仅 TiFlash 情况表现更好。

Q12 主要包含两个表连接，但在每种存储类型中采用不同的物理实现。在仅限 TiKV 的情况下，它使用索引连接，从表 ORDER LINE 中扫描几个合格的元组，并使用索引查找表 OORDER。索引读取器的成本要低得多，以至于它优于仅使用 TiFlash 的情况下的哈希联接（后者从两个表中扫描所需的列）。同时使用 TiKV 和 TiFlash 时，成本会进一步降低，因为它使用更便宜的索引连接，从 TiFlash 扫描 ORDER LINE，并使用 TiKV 中的索引查找 OORDER。在 TiKV 和 TiFlash 情况下，读取列存储将仅 TiKV 情况的执行时间减少一半。

在 Q22 中，其 *exists()* 子查询被转换为反半连接。它在仅限 TiKV 的情况下使用索引联接，在仅限 TiFlash 的情况下使用哈希联接。但与 Q12 中的执行不同，使用索引连接比哈希连接更昂贵。当从 TiFlash 获取内表并使用 TiKV 的索引查找外表时，索引连接的成本会降低。因此，TiKV 和 TiFlash 情况再次花费最少的时间。

Q8 更复杂。它包含与九个表的联接。在仅 TiKV 的情况下，它需要两个索引合并连接和六个哈希连接，并使用索引查找两个表（CUSTOMER 和 OORDER）。该计划需要 1.13 秒，优于仅 TiFlash 情况下的 8 个哈希连接（花费 1.64 秒）。在 TiKV 和 TiFlash 情况下，它的开销进一步减少，除了在六个哈希连接中从 TiFlash 扫描数据之外，物理计划几乎没有变化。此改进将执行时间缩短至 0.55 秒。在这三个查询中，仅使用 TiKV 或 TiFlash 会获得不同的性能，将它们结合起来可获得最佳结果。

对于 Q1、Q4、Q6、Q11、Q13、Q14 和 Q19，仅 TiFlash 情况的性能优于仅 TiKV 情况，并且 TiKV 和 TiFlash 情况与仅 TiFlash 情况具有相同的性能。真正的

儿子们对这七个问题的看法各不相同。Q1 和 Q6 主要由单表上的聚合组成，因此在 TiFlash 中的列存储上运行花费的时间较少，是最佳选择。这些结果凸显了之前工作中描述的柱状存储的优势。Q4 和 Q11 在每种情况下使用相同的物理计划单独执行。然而，从 TiFlash 扫描数据比 TiKV 便宜，因此仅 TiFlash 情况下的执行时间更少，也是最佳选择。Q13、Q14 和 Q19 各包含一个两表联接，该联接以哈希联接的形式实现。虽然仅 TiKV 的情况在探测哈希表时采用索引读取器，但它也比从 TiFlash 扫描数据更昂贵。

Q9 是多连接查询。在仅限 TiKV 的情况下，它会使用索引对某些表进行索引合并连接。它比在 TiFlash 上进行哈希连接便宜，因此它成为最佳选择。Q7、Q20 和 Q21 产生类似的结果，但由于空间有限，它们被省略。22 个 TPC-H 查询中的其余 8 个在三种存储设置中具有相当的性能。

此外，我们使用 CH-benCHmark 的 500 个仓库的 22 个分析查询，将 TiSpark 与 SparkSQL、PrestoDB 和 Greenplum 进行比较。每个数据库安装在六台服务器上。对于 SparkSQL 和 PrestoDB，数据在 Hive 中存储为列式 parquet 文件。图 9 比较了这些系统的性能。TiSpark 的性能与 SparkSQL 相当，因为它们使用相同的引擎。性能差距相当小，主要来自于访问不同的存储系统：扫描压缩的 parquet 文件更便宜，因此 SparkSQL 通常优于 TiSpark。然而，在某些情况下，TiSpark 可以将更多计算推送到存储层，这种优势会被抵消。TiSpark 与 PrestoDB 和 Greenplum 的比较是 SparkSQL（TiSpark 的底层引擎）与其他两个引擎的比较。然而，这超出了本文的范围，我们不会详细讨论它。

6.4 HTAP 性能

除了研究事务处理（TP）和分析处理（AP）性能之外，我们还使用基于整个 CH-benCHmark 的混合作负载以及单独的事务客户端（TC）和分析客户端（AC）来评估 TiDB。这些实验在 100 个仓库中进行。数据加载到 TiKV 中，同时复制到 TiFlash 中。TiKV 部署在三台服务器上，由 TiDB SQL 引擎实例访问。TiFlash 部署在另外三台服务器上，并与 TiSpark 实例搭配。此配置分别为分析查询和事务查询提供服务。每次运行 10 分钟，其中有 3 分钟的热身时间。我们测量了 TP 和 AP 工作负载的吞吐量和平均延迟。

图 10(a) 和 10(b) 分别显示了具有不同数量的 TP 客户端和 AP 客户端的事务的吞吐量和平均延迟。吞吐量随着 TP 客户端的增加而增加，但在略少于 512 个客户端时达到最大值。和

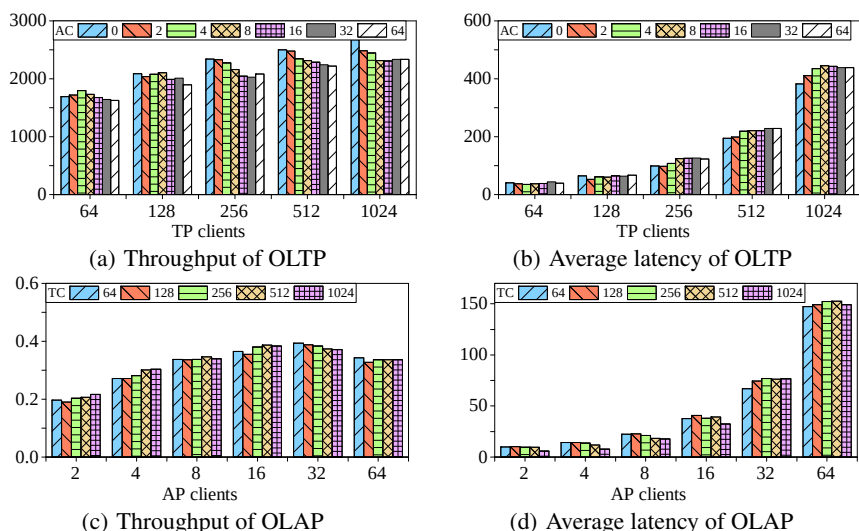


Figure 10: HTAP performance of TiDB

Table 4: The count distribution of visibility delay

W	Time (ms)	AC=2 TC=64	AC=2 TC=1024	AC=32 TC=64	AC=32 TC=1024
10	< 100	95.98%	96.22%	84.13%	90.96%
	< 500	99.43%	99.36%	98.98%	98.45%
	< 1000	99.62%	99.90%	99.74%	99.49%
100	< 100	82.98%	86.31%	48.51%	48.77%
	< 500	95.50%	97.69%	71.24%	72.06%
	< 1000	98.64%	99.58%	84.82%	87.17%

相同数量的 TP 客户端，与没有 AP 客户端相比，更多的分析处理客户端会使 TP 吞吐量最多降低 10%。这证实了 TiKV 和 TiFlash 之间的日志复制实现了高度隔离，特别是与 6.6 节中 MemSQL 的性能对比。该结果与[24]中的结果类似。

事务的平均延迟会增加且没有上限。这是因为即使更多的客户端发出更多的请求，它们也不能立即完成，必须等待。等待时间导致延迟增加。

图 10(c) 和 10(d) 中显示的类似吞吐量和延迟结果表明了 TP 对 AP 请求的影响。在 16 个 AP 客户端下，AP 吞吐量很快就会达到最大值，因为 AP 查询成本高昂并且会争夺资源。这种争用会降低更多 AP 客户端的吞吐量。在 AP 客户端数量相同的情况下，吞吐量几乎保持不变，最多只下降 5%。这表明 TP 不会显著影响 AP 执行。分析查询的平均延迟增加是由于更多客户端的等待时间更长而导致的。

6.5 日志复制延迟

为了实现实时分析处理，事务更新应该立即对 TiFlash 可见。这种数据新鲜度是由 TiKV 和 TiFlash 之间的日志复制延迟决定的。我们在使用不同数量的事务客户端和分析客户端运行 CH-benCHmark 时测量日志复制时间。我们记录运行 CH-benCHmark 的 10 分钟内每次复制的延迟，并每 10 秒计算一次平均延迟。我们还计算了 10 分钟内日志复制延迟的分布，如表 4 所示。

如图 11 (a) 所示，10 个仓库的日志复制延迟始终小于 300 毫秒，并且大多数延迟都小于 100 毫秒。图 11(b) 显示，随着仓库数量增加 100 个，延迟会增加；大多数小于 1000 毫秒。表 4 给出了更精确的

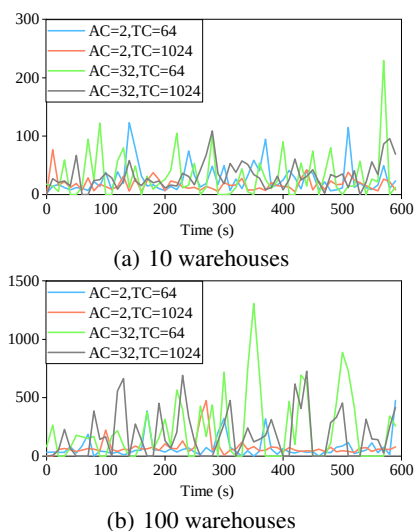


Figure 11: Visibility delay of log replication

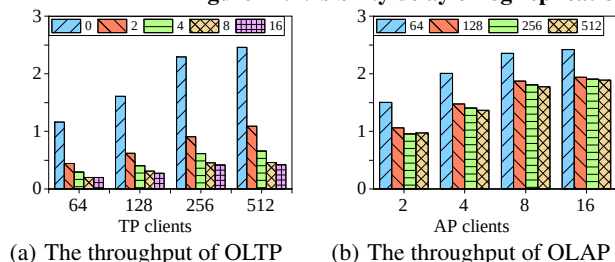


图 12: MemSQL 的 HTAP 性能

细节。对于 10 个仓库，几乎 99% 的查询成本低于 500 毫秒，无论客户端设置如何。对于 100 个仓库，在 2 个和 32 个分析客户端的情况下，大约 99% 和 85% 的查询分别花费不到 1 000 毫秒。这些指标强调 TiDB 可以保证 HTAP 工作负载上约一秒的数据新鲜度。

比较图 11(a) 和图 11(b) 时，我们观察到延迟时间与数据大小有关。仓库越多，延迟越大，因为数据越多，需要同步的日志就越多。此外，延迟还取决于分析请求的数量，但由于事务客户端的数量而受到的影响较小。这可以在图 11(b) 中清楚地看到。三十二个 AC 比两个 AC 导致更多的延迟。但对于相同数量的分析客户端，延迟差别并不大。我们在表 4 中显示了更精确的结果。对于 100 个仓库和两个 AC，超过 80% 的查询花费的时间少于 100 毫秒，但对于 32 个 AC，不到 50% 的查询花费的时间少于 100 毫秒。这是因为更多的分析查询会导致更高频率的日志复制。

6.6 与 MemSQL 的比较

我们使用 CH-benC -Hmark 将 TiDB 与 MemSQL 7.0 进行比较。该实验旨在强调最先进的 HTAP 系统的隔离问题，而不是 OLTP 和 OLAP 性能。MemSQL 是一种分布式关系数据库，可大规模处理事务和实时分析。MemSQL 部署在六台服务器上：一台主服务器、一台聚合器和四台叶子服务器。我们将 100 个仓库加载到 MemSQL 中，并使用不同数量的 AP 和 TP 客户端运行基准测试。基准测试运行了 10 分钟，并有 5 分钟的预热时间。

与图 10 相比，图 12 说明工作负载干扰对 MemSQL 的性能有显著影响。特别是，随着 AP 客户端数量的增加，交易量

吞吐量显著降低，下降了五倍以上。AP吞吐量也会随着TP客户端的增加而降低，但这种效果并不明显，因为事务查询不需要分析查询的大量资源。

7. 相关工作

构建 HTAP 系统的常见方法是：从现有数据库演变、扩展开源分析系统或从头开始构建。TiDB 是从头开始构建的，在架构、数据起源、计算引擎和一致性保证方面与其他系统不同。

从现有数据库发展而来。成熟的数据库可以在现有产品的基础上提供HTAP解决方案，并且特别注重加速分析查询。他们采用定制方法来单独实现数据一致性和高可用性。相比之下，TiDB 自然受益于 Raft 中的日志复制来实现数据一致性和高可用性。

Oracle [19] 于 2014 年推出了内存数据库选项，作为业界第一个双格式内存 RDBMS。该选项旨在打破分析查询工作负载中的性能障碍，而不影响（甚至提高）常规事务工作负载的性能。列式存储是只读快照，在某个时间点保持一致，并使用完全在线重新填充机制进行更新。Oracle 后来的工作 [27] 展示了其分布式架构的高可用性方面，并提供容错分析查询执行。

SQL Server [21] 将两个专用存储引擎集成到其核心中：用于分析工作负载的 Apollo 列存储引擎和用于事务工作负载的 Hekaton 内存引擎。数据迁移任务定期将 Hekaton 表尾部的数据复制到压缩列存储中。SQL Server 使用列存储索引和批处理来有效地处理分析查询，并利用 SIMD [15] 进行数据扫描。

SAP HANA 支持有效评估单独的 OLAP 和 OLTP 查询，并为每个查询使用不同的数据组织。为了扩展 OLAP 性能，它将行存储数据异步复制到分布在服务器集群上的列式存储 [22]。该方法为 MVCC 数据提供亚秒级可见性。然而，它需要付出大量的努力来处理错误并保持数据的一致性。重要的是，事务引擎缺乏高可用性，因为它仅部署在单个节点上。

改造开源系统。Apache Spark 是一个用于数据分析的开源框架。它需要一个事务模块来实现HTAP。下面列出的许多系统都遵循这个想法。TiDB 并不深深依赖于 Spark，因为 TiSpark 是一个扩展。TiDB 是一个独立的 HTAP 数据库，没有 TiSpark。

Wildfire [10, 9] 基于 Spark 构建了一个 HTAP 引擎。它在同一列数据组织（即 Parquet）上处理分析请求和事务请求。它采用最后写入获胜语义进行并发更新，并采用快照隔离进行读取。为了实现高可用性，分片日志无需共识算法的帮助即可复制到多个节点。分析查询和事务查询可以在不同的节点上处理；但是，处理最新更新有明显的延迟。Wildfire 对大规模 HTAP 工作负载使用统一的多版本和多区域索引方法 [23]。

SnappyData [25] 为 OLTP、OLAP 和流分析提供了一个统一的平台。它将用于高吞吐量分析的计算引擎 (Spark) 与横向扩展的内存中事务存储 (GemFire) 集成在一起。最近的更新以行格式存储，然后老化为列格式以供分析查询。事务遵循使用 GemFire 的 Paxos 实现的 2PC 协议，以确保整个集群的共识和一致的视图。

从头开始构建。许多新的 HTAP 系统已经研究了混合工作负载的不同方面，其中包括利用内存计算来提高性能、优化数据存储和可用性。与 TiDB 不同，它们无法同时提供高可用性、数据一致性、可扩展性、数据新鲜度和隔离性。

MemSQL [3] 拥有一个用于可扩展内存中 OLTP 和快速分析查询的引擎。MemSQL 可以以行或列格式存储数据库表。它可以将部分数据保留为行格式，并将其转换为列格式，以便在将数据写入磁盘时进行快速分析。它将重复查询编译为低级机器代码以加速分析查询，并使用许多无锁结构来辅助事务处理。但是，在运行 HTAP 工作负载时，它无法为 OLAP 和 OLTP 提供独立的性能。

HyPer [18] 使用操作系统的 fork 系统调用为分析工作负载提供快照隔离。其较新版本采用 MVCC 实现来提供可串行性、快速事务处理和快速扫描。ScyPer [26] 扩展了 HyPer，通过使用逻辑或物理重做日志传播更新来大规模评估远程副本上的分析查询。

BatchDB [24] 是一个专为 HTAP 工作负载设计的内存数据库引擎。它依赖于具有专用副本的主从复制，每个副本都针对特定的工作负载类型（即 OLTP 或 OLAP）进行了优化。它最大限度地减少了事务引擎和分析引擎之间的负载交互，从而能够在 HTAP 工作负载的严格 SLA 下对新数据进行实时分析。请注意，它对行格式副本执行分析查询，并且不保证高可用性。

基于谱系的数据存储 (L-Store) [35]通过引入更新友好的、基于谱系的存储架构，将实时分析和事务查询处理结合在单个统一引擎中。该存储在本机多版本列式存储模型上启用无争用更新机制，以便延迟且独立地将稳定数据从写优化列式格式转移到读优化列式布局。

Peloton [31] 是一个自动驱动的 SQL 数据库管理系统。它尝试在运行时针对 HTAP 工作负载调整数据来源 [8]。它使用无锁、多版本并发控制来支持实时分析。然而，它在设计上是一个单节点内存数据库。

Cockroach DB [38] 是一个分布式 SQL 数据库，提供高可用性、数据一致性、可扩展性和隔离性。与 TiDB 一样，它构建在 Raft 算法之上并支持分布式事务。它提供了更强的隔离属性：可串行化，而不是快照隔离。但是，它不支持专用的 OLAP 或 HTAP 功能。

八、结论

我们推出了一个可用于生产的 HTAP 数据库：TiDB。TiDB 构建在 TiKV 之上，TiKV 是一种分布式、基于行的存储，使用 Raft 算法。我们引入柱状学习器进行实时分析，它从 TiKV 异步复制日志，并将行格式数据转换为列格式。TiKV 和 TiFlash 之间的这种日志复制提供了实时数据一致性，且开销很小。TiKV 和 TiFlash 可以部署在单独的物理资源上，以有效地处理事务和分析查询。在扫描表进行事务和分析查询时，TiDB 可以优化选择要访问的它们。实验结果表明 TiDB 在 HTAP 基准 CH-benCHmark 下表现良好。TiDB 提供了将 NewSQL 系统演变成 HTAP 系统的通用解决方案。

9. 参考文献

- [1] 点击屋. <https://clickhouse.tech>. [2] LZ4. <https://github.com/lz4/lz4>. [3] 内存SQL. <https://www.memsql.com>. [4] 镶木地板. <https://parquet.apache.org>. [5] RocksDB. <https://rocksdb.org>. [6] 系统基准. <https://github.com/akopytov/sysbench>. [7] TiDB. <https://github.com/pingcap/tidb>. [8] J. Arulraj, A. Pavlo 和 P. Menon. 在混合工作负载的行存储和列存储之间架起桥梁. 在 *SIGMOD* 中, 第 583-598 页. ACM, 2016. [9] R. Barber, C. Garcia-Arellano, R. Grosman, R. Müller 等人. 新一代大数据应用的不断发展的数据库. 在 *CIDR* 中. www.cidrdb.org, 2017. [10] R. Barber, M. Huras, G. M. Lohman, C. Mohan 等. 野火: 并发的炽热数据摄取和分析. 在 *SIGMOD* 中, 第 2077-2080 页. ACM, 2016. [11] R. 卡特尔. 可扩展的 SQL 和 NoSQL 数据存储. *SIGMOD Rec.*, 39(4): 12-27, 2010. [12] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes 和 R. Gruber. Bigtable: 结构化数据的分布式存储系统. 在 *OSDI* 中, 第 205-218 页. USENIX 协会, 2006 年. [13] R. L. Cole, F. Funke, L. Giakoumakis, W. Guy 等人. 混合工作负载 CH-benchmark. *DBTest 2011*, 第 8 页. ACM, 2011 年. [14] J. C. Corbett, J. Dean, M. Epstein, A. Fikes 等人. Spanner: 谷歌的全球分布式数据库. *ACM Trans. Comput. Syst.*, 31(3):8:1-8:22, 2013. [15] Z. Fang, B. Cheng, C. Weng. 交错多矢量化. *PVLDB*, 13(3):226-238, 2019. [16] A. Floratou, U. F. Minhas, 和 F. Özcan. SQL-on-Hadoop: 回到无共享数据库架构. *PVLDB*, 7(12): 1295-1306, 2014. [17] G. 格雷夫. Volcano - 可扩展的并行查询评估系统. *IEEE Trans. Knowl. Data Eng.*, 6(1):120-135, 1994. [18] A. Kemper 和 T. Neumann. HyPer: 基于虚拟内存快照的混合 OLTP+OLAP 主存数据库系统. 在 *ICDE* 中, 第 195-206 页. IEEE 计算机协会, 2011. [19] T. Lahiri, S. Chavan, M. Colgan, D. Das, A. Ganesh 等人. Oracle Database In-Memory: 双格式内存数据库. 在 *ICDE* 中, 第 1253-1258 页. IEEE 计算机协会, 2015 年. [20] L. Lamping. 兼职议会. *ACM Trans. Comput. Syst.*, 16(2):133-169, 1998. [21] P. Larson, A. Birka, E. N. Hanson, W. Huang, M. Nowakiewicz 和 V. Papadimos. 使用 SQL Server 进行实时分析处理. *PVLDB*, 8(12): 1740-1751, 2015. [22] J. Lee, S. Moon, K. H. Kim, D. H. Kim, S. K. Cha, W. Han, C. G. Park, H. J. Na 和 J. Lee. 跨并行复制 SAP HANA 中用于横向扩展混合 OLTP/OLAP 工作负载的格式. *PVLDB*, 10(12):1598-1609, 2017. [23] C. Luo, P. Tözün, Y. Tian, R. Barber, et al. Umzi: 大规模 HTAP 的统一多区域索引. 在 *EDBT* 中, 第 1-12 页. OpenProceedings.org, 2019. [24] D. Makreshanski, J. Giceva, C. Barthels 和 G. Alonso. BatchDB: 交互式应用程序的混合 OLTP+OLAP 工作负载的高效隔离执行. 在 *SIGMOD* 中, 第 37-50 页. ACM, 2017. [25] B. Mozafari, J. Ramnarayan, S. Menon, Y. Mahajan, S. Chakraborty, H. Bhanawat 和 K. Bachhav. SnappyData: 用于流、事务和交互式分析的统一集群. 在 *CIDR* 中. www.cidrdb.org, 2017. [26] T. Mühlbauer, W. Rödiger, A. Reiser, A. Kemper 和 T. Neumann. ScyPer: 用于可扩展实时分析的混合 OLTP 和 OLAP 分布式主内存数据库系统. 在 *DBIS* 中, *LNI* 的第 P-214 卷, 第 499-502 页. GI, 2013. [27] N. Mukherjee, S. Chavan, M. Colgan, M. Gleeson, X. He 等人. 使用分布式 Oracle 内存数据库进行容错实时分析. 在 *ICDE* 中, 第 1298-1309 页. IEEE 计算机协会, 2016 年. [28] P. E. O'Neil, E. Cheng, D. Gawlick 和 E. J. O'Neil. 日志结构合并树 (LSM 树). *Acta Inf.*, 33(4): 351-385, 1996. [29] D. Ongaro 和 J. K. Ousterhout. 寻找一种可理解的共识算法. 在 *USENIX ATC* 中, 第 305-319 页. USENIX 协会, 2014 年. [30] F. Özcan, Y. Tian 和 P. Tözün. 混合事务/分析处理: 一项调查. 在 *SIGMOD* 中, 第 1771-1775 页. ACM, 2017. [31] A. Pavlo, G. Angulo, J. Arulraj, H. Lin, J. Lin 等人. 自驱动数据库管理系统. 在 *CIDR* 中. www.cidrdb.org, 2017. [32] A. Pavlo 和 M. Aslett. NewSQL 有什么真正的新功能? *SIGMOD*, 45(2):45-55, 2016. [33] D. Peng 和 F. Dabek. 使用分布式事务和通知进行大规模增量处理. 在 *OSDI* 中, 第 251-264 页. USENIX 协会, 2010. [34] I. Psaroudakis, F. Wolf, N. May, T. Neumann, A. Böhm, A. Ailamaki 和 K. Sattler. 扩展混合工作负载: 数据新鲜度、灵活性和调度之战. 在 *TPCTC*, 第 8904 卷, 第 97-112 页. 施普林格, 2014. [35] M. Sadoghi, S. Bhattacherjee, B. Bhattacherjee 和 M. C. Canim. L-Store: 实时 OLTP 和 OLAP 系统. 在 *EDBT* 中, 第 540-551 页. OpenProceedings.org, 2018. [36] S. Sivasubramanian. Amazon dynamoDB: 无缝可扩展的非关系数据库服务. 在 *SIGMOD* 中, 第 729-730 页. ACM, 2012. [37] M. Stonebraker 和 U. Çetintemel. “一刀切”: 一个时代已经过去又过去的想法 (摘要). 在 *ICDE* 中, 第 2-11 页. IEEE 计算机协会, 2005 年. [38] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis 等人. CockroachDB: 弹性地理分布式 SQL 数据库. 在 *SIGMOD* 中, 第 1493-1509 页. 美国CM, 2020.